PROJECT REVIEW

Reg. No & Name

- Ansari Moinuddin M - 26MAG0043
- Mohammad Saad - 26MAG0061
- Iman Huda - 26MAG0095

Title of project: Intelligent Resume Analysis and Job Recommendation System using Retrieval-Augmented Generation and Large Language Models

1. Proposed Work

The proposed work, **VECTOR (AI Resume Intelligence)**, is an advanced AI-powered Applicant Tracking System (ATS) and Resume Analyzer.

Improvement in Existing Work

Traditional resume screening systems rely almost entirely on rigid **keyword matching** (Boolean searches). If a job requires "Frontend Development" and the resume says "React.js," older systems often fail to connect the two. Furthermore, traditional systems are a "black box", they output a score without explaining *why* a candidate was selected or rejected.

This project improves upon existing systems by introducing **Semantic Search** and **Explainable AI**. Instead of just looking for exact words, the system understands the *meaning* and *context* of the resume. It extracts structured data (skills, years of experience, entities) using Natural Language Processing (NLP), embeds the text into a vector space to find true conceptual matches, and uses a Large Language Model (LLM) to explain its reasoning in plain English.

2. Comparison Between Previous Model and Current Model

Feature	Previous Model (Traditional ATS)	Current Model (VECTOR AI Analyzer)
Matching Mechanism	Exact keyword intersection (Regex / Boolean).	Semantic similarity using `sentence-transformers` and vector embeddings.
Context Awareness	Low. Cannot differentiate between "used Python for 5 years" vs "read about Python".	High. Uses `spaCy` NLP to accurately extract skills tied to years of experience.
Database	Relational databases storing raw text.	**ChromaDB** (Vector Database) for fast, similarity-based mathematical nearest-neighbor search.
Explainability	None. Outputs an arbitrary percentage score.	High. Uses **Ollama (LLM)** to generate detailed paragraphs explaining matched skills and highlighting missing skill gaps.

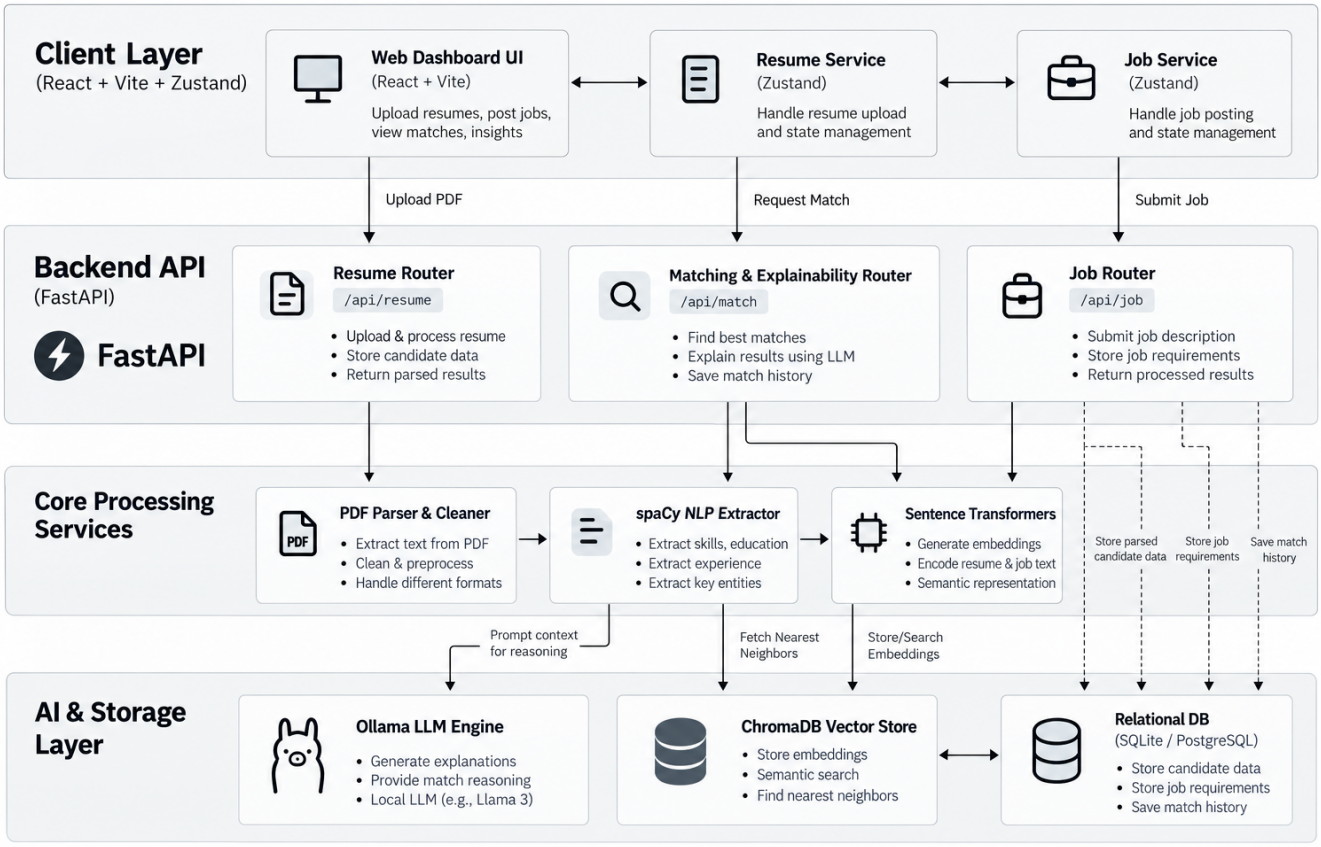
Feature	Previous Model (Traditional ATS)	Current Model (VECTOR AI Analyzer)
Candidate Feedback	Usually none (candidates are silently rejected).	Generates actionable "Resume Improvement Plans" to help candidates bridge skill gaps.

3. Result

The implementation of this architecture yields the following results:

- **Significantly Higher Accuracy:** Reduces false negatives (good candidates rejected due to missing exact keywords) by understanding semantic equivalents.
- **Transparent Recruiting:** Recruiters no longer have to blindly trust an AI score; they receive a human-readable justification for every match.
- **Reduced Bias:** By evaluating pure semantic skill alignment rather than formatting or keyword stuffing, the evaluation becomes fairer and more objective.
- **Real-time Processing:** The decoupled architecture (FastAPI backend + React frontend) allows for sub-second vector searches and instantaneous dashboard updates.

4. Architecture Diagram



5. Explanation

The architecture is divided into decoupled micro-layers to ensure scalability and performance:

1. **Frontend (React/Vite):** Acts as the presentation layer. It provides an intuitive dashboard for recruiters to upload resumes, paste job descriptions, and view side-by-side analytical comparisons.

2. **Backend API (FastAPI):** Acts as the bridge. FastAPI provides high-performance, asynchronous endpoints that route data between the user interface and the heavy machine-learning models.
3. **NLP & Embeddings Pipeline:** When a resume is uploaded, text is extracted via PyMuPDF. It is then passed to `spaCy` to recognize named entities (like technologies and certifications). `sentence-transformers` then converts the cleaned text into numerical vectors.
4. **Vector Database (ChromaDB):** Instead of matching strings, ChromaDB mathematically measures the "distance" between the Resume Vector and the Job Description Vector. A shorter distance means a higher match score.
5. **Explainability Engine (Ollama/LLM API-key):** The system passes the top matching results, along with the required skills, to a local LLM. The LLM acts as the "reasoning" engine, outputting a human-readable summary of why the candidate fits the role and what skills they are currently lacking.
6. **Proposed Persistence (Relational DB):** Adding SQLite/PostgreSQL alongside ChromaDB allows the system to permanently store candidates, track match history over time, and build long-term talent pools (rather than relying on temporary frontend session storage).